

```

        int horsepower;
        double weight;
    public:
        SnowMobile(int h, double w), Vehicle(h)
        { horsepower = h; }
        ~SnowMobile();
};

56. class Table : public Furniture
{
    protected:
        int numSeats;
    public:
        Table(int n) : Furniture(numSeats)
        { numSeats = n; }
        ~Table();
};

57. class Tank : public Cylinder
{
    private:
        int fuelType;
        double gallons;
    public:
        Tank();
        ~Tank();
        void setContents(double);
        void setContents(double);
};

58. class Three : public Two : public One
{
    protected:
        int x;
    public:
        Three(int a, int b, int c), Two(b), Three(c)
        { x = a; }
        ~Three();
};

```

Programming Challenges



VideoNote
Solving the
Employee and
Production-
Worker Classes
Problem

1. Employee and ProductionWorker Classes

Design a class named `Employee`. The class should keep the following information in

- Employee name
- Employee number
- Hire date

Write one or more constructors and the appropriate accessor and mutator functions for the class.

Next, write a class named `ProductionWorker` that is derived from the `Employee` class. The `ProductionWorker` class should have member variables to hold the following information:

- Shift (an integer)
- Hourly pay rate (a double)

The workday is divided into two shifts: day and night. The shift variable will hold an integer value representing the shift that the employee works. The day shift is shift 1, and the night shift is shift 2. Write one or more constructors and the appropriate accessor and mutator functions for the class. Demonstrate the classes by writing a program that uses a `ProductionWorker` object.

2. `ShiftSupervisor` Class

In a particular factory a shift supervisor is a salaried employee who supervises a shift. In addition to a salary, the shift supervisor earns a yearly bonus when his or her shift meets production goals. Design a `ShiftSupervisor` class that is derived from the `Employee` class you created in Programming Challenge 1. The `ShiftSupervisor` class should have a member variable that holds the annual salary and a member variable that holds the annual production bonus that a shift supervisor has earned. Write one or more constructors and the appropriate accessor and mutator functions for the class. Demonstrate the class by writing a program that uses a `ShiftSupervisor` object.

3. `TeamLeader` Class

In a particular factory, a team leader is an hourly paid production worker who leads a small team. In addition to hourly pay, team leaders earn a fixed monthly bonus. Team leaders are required to attend a minimum number of hours of training per year. Design a `TeamLeader` class that extends the `ProductionWorker` class you designed in Programming Challenge 1. The `TeamLeader` class should have member variables for the monthly bonus amount, the required number of training hours, and the number of training hours that the team leader has attended. Write one or more constructors and the appropriate accessor and mutator functions for the class. Demonstrate the class by writing a program that uses a `TeamLeader` object.

4. Time Format

In Program 15-20, the file `Time.h` contains a `Time` class. Design a class called `MilTime` that is derived from the `Time` class. The `MilTime` class should convert time in military (24-hour) format to the standard time format used by the `Time` class. The class should have the following member variables:

- `milHours`: Contains the hour in 24-hour format. For example, 1:00 pm would be stored as 1300 hours, and 4:30 pm would be stored as 1630 hours.
- `milSeconds`: Contains the seconds in standard format.

The class should have the following member functions:

- `Constructor`: The constructor should accept arguments for the hour and seconds, in military format. The time should then be converted to standard time and stored in the `hours`, `min`, and `sec` variables of the `Time` class.
- `setTime`: Accepts arguments to be stored in the `milHours` and `milSeconds` variables. The time should then be converted to standard time and stored in the `hours`, `min`, and `sec` variables of the `Time` class.

`getHour:` Returns the hour in military format.

`getStandHr:` Returns the hour in standard format.

Demonstrate the class in a program that asks the user to enter the time in military format. The program should then display the time in both military and standard format.

Input Validation: The `MilTime` class should not accept hours greater than 2359, or less than 0. It should not accept seconds greater than 59 or less than 0.

5. Time Clock

Design a class named `TimeClock`. The class should be derived from the `MilTime` class you designed in Programming Challenge 4. The class should allow the programmer to pass two times to it: starting time and ending time. The class should have a member function that returns the amount of time elapsed between the two times. For example, if the starting time is 900 hours (9:00 am), and the ending time is 1300 hours (1:00 pm), the elapsed time is 4 hours.

Input Validation: The class should not accept hours greater than 2359 or less than 0.

6. Essay class

Design an `Essay` class that is derived from the `GradedActivity` class presented in this chapter. The `Essay` class should determine the grade a student receives on an essay. The student's essay score can be up to 100, and is determined in the following manner:

- Grammar: 30 points
- Spelling: 20 points
- Correct length: 20 points
- Content: 30 points

Demonstrate the class in a simple program.

7. `PersonData` and `CustomerData` classes

Design a class named `PersonData` with the following member variables:

- `lastName`
- `firstName`
- `address`
- `city`
- `state`
- `zip`
- `phone`

Write the appropriate accessor and mutator functions for these member variables.

Next, design a class named `CustomerData`, which is derived from the `PersonData` class. The `CustomerData` class should have the following member variables:

- `customerNumber`
- `mailingList`

The `customerNumber` variable will be used to hold a unique integer for each customer. The `mailingList` variable should be a `bool`. It will be set to `true` if the customer wishes to be on a mailing list, or `false` if the customer does not wish to be on a mailing list. Write appropriate accessor and mutator functions for these member variables. Demonstrate an object of the `CustomerData` class in a simple program.

8. PreferredCustomer Class

A retail store has a preferred customer plan where customers may earn discounts on all their purchases. The amount of a customer's discount is determined by the amount of the customer's cumulative purchases in the store.

- When a preferred customer spends \$500, he or she gets a 5% discount on all future purchases.
- When a preferred customer spends \$1,000, he or she gets a 6% discount on all future purchases.
- When a preferred customer spends \$1,500, he or she gets a 7% discount on all future purchases.
- When a preferred customer spends \$2,000 or more, he or she gets a 10% discount on all future purchases.

Design a class named `PreferredCustomer`, which is derived from the `CustomerData` class you created in Programming Challenge 7. The `PreferredCustomer` class should have the following member variables:

- `purchasesAmount` (a double)
- `discountLevel` (a double)

The `purchasesAmount` variable holds the total of a customer's purchases to date. The `discountLevel` variable should be set to the correct discount percentage, according to the store's preferred customer plan. Write appropriate member functions for this class and demonstrate it in a simple program.

Input Validation: Do not accept negative values for any sales figures.

9. File Filter

A file filter reads an input file, transforms it in some way, and writes the results to an output file. Write an abstract file filter class that defines a pure virtual function for transforming a character. Create one derived class of your file filter class that performs encryption, another that transforms a file to all uppercase, and another that creates an unchanged copy of the original file. The class should have the following member function:

```
void doFilter(ifstream &in, ofstream &out)
```

This function should be called to perform the actual filtering. The member function for transforming a single character should have the prototype:

```
char transform(char ch)
```

The encryption class should have a constructor that takes an integer as an argument and uses it as the encryption key.

10. File Double-Spacer

Create a derived class of the abstract filter class of Programming Challenge 9 that double-spaces a file: that is, it inserts a blank line between any two lines of the file.

11. Course Grades

In a course, a teacher gives the following tests and assignments:

- A **lab activity** that is observed by the teacher and assigned a numeric score.
- A **pass/fail exam** that has 10 questions. The minimum passing score is 70.
- An **essay** that is assigned a numeric score.
- A **final exam** that has 50 questions.

Write a class named `CourseGrades`. The class should have a member named `grades` that is an array of `GradedActivity` pointers. The `grades` array should have four elements, one for each of the assignments previously described. The class should have the following member functions:

<code>setLab:</code>	This function should accept the address of a <code>GradedActivity</code> object as its argument. This object should already hold the student's score for the lab activity. Element 0 of the <code>grades</code> array should reference this object.
<code>setPassFailExam:</code>	This function should accept the address of a <code>PassFailExam</code> object as its argument. This object should already hold the student's score for the pass/fail exam. Element 1 of the <code>grades</code> array should reference this object.
<code>setEssay:</code>	This function should accept the address of an <code>Essay</code> object as its argument. (See Programming Challenge 6 for the <code>Essay</code> class. If you have not completed Programming Challenge 6, use a <code>GradedActivity</code> object instead.) This object should already hold the student's score for the essay. Element 2 of the <code>grades</code> array should reference this object.
<code>setPassFailExam:</code>	This function should accept the address of a <code>FinalExam</code> object as its argument. This object should already hold the student's score for the final exam. Element 3 of the <code>grades</code> array should reference this object.
<code>print:</code>	This function should display the numeric scores and grades for each element in the <code>grades</code> array.

Demonstrate the class in a program.

12. **Ship, CruiseShip, and CargoShip** Classes

Design a `Ship` class that has the following members:

- A member variable for the name of the ship (a string)
- A member variable for the year that the ship was built (a string)
- A constructor and appropriate accessors and mutators
- A virtual `print` function that displays the ship's name and the year it was built.

Design a `CruiseShip` class that is derived from the `Ship` class. The `CruiseShip` class should have the following members:

- A member variable for the maximum number of passengers (an `int`)
- A constructor and appropriate accessors and mutators
- A `print` function that overrides the `print` function in the base class. The `CruiseShip` class's `print` function should display only the ship's name and the maximum number of passengers.

Design a `CargoShip` class that is derived from the `Ship` class. The `CargoShip` class should have the following members:

- A member variable for the cargo capacity in tonnage (an `int`).
- A constructor and appropriate accessors and mutators.
- A `print` function that overrides the `print` function in the base class. The `CargoShip` class's `print` function should display only the ship's name and the ship's cargo capacity.

Demonstrate the classes in a program that has an array of `Ship` pointers. The array elements should be initialized with the addresses of dynamically allocated `Ship`, `CruiseShip`, and `CargoShip` objects. (See Program 15-14, lines 17 through 22, for an example of how to do this.) The program should then step through the array, calling each object's `print` function.

13. Pure Abstract Base Class Project

Define a pure abstract base class called `BasicShape`. The `BasicShape` class should have the following members:

Private Member Variable:

`area`, a double used to hold the shape's area.

Public Member Functions:

`getArea`. This function should return the value in the member variable `area`.

`calcArea`. This function should be a pure virtual function.

Next, define a class named `Circle`. It should be derived from the `BasicShape` class. It should have the following members:

Private Member Variables:

`centerX`, a long integer used to hold the x coordinate of the circle's center.

`centerY`, a long integer used to hold the y coordinate of the circle's center.

`radius`, a double used to hold the circle's radius.

Public Member Functions:

constructor—accepts values for `centerX`, `centerY`, and `radius`. Should call the overridden `calcArea` function described below.

`getCenterX`—returns the value in `centerX`.

`getCenterY`—returns the value in `centerY`.

`calcArea`—calculates the area of the circle ($\text{area} = 3.14159 * \text{radius} * \text{radius}$) and stores the result in the inherited member `area`.

Next, define a class named `Rectangle`. It should be derived from the `BasicShape` class. It should have the following members:

Private Member Variables:

`width`, a long integer used to hold the width of the rectangle.

`length`, a long integer used to hold the length of the rectangle.

Public Member Functions:

constructor—accepts values for `width` and `length`. Should call the overridden `calcArea` function described below.

`getWidth`—returns the value in `width`.

`getLength`—returns the value in `length`.

`calcArea`—calculates the area of the rectangle ($\text{area} = \text{length} * \text{width}$) and stores the result in the inherited member `area`.

After you have created these classes, create a driver program that defines a `Circle` object and a `Rectangle` object. Demonstrate that each object properly calculates and reports its area.

Group Project

14. Bank Accounts

This program should be designed and written by a team of students. Here are some suggestions:

- One or more students may work on a single class.
- The requirements of the program should be analyzed so each student is given about the same work load.
- The parameters and return types of each function and class member function should be decided in advance.
- The program will be best implemented as a multi-file program.

Design a generic class to hold the following information about a bank account:

Balance

Number of deposits this month

Number of withdrawals

Annual interest rate

Monthly service charges

The class should have the following member functions:

Constructor: Accepts arguments for the balance and annual interest rate.

deposit: A virtual function that accepts an argument for the amount of the deposit. The function should add the argument to the account balance. It should also increment the variable holding the number of deposits.

withdraw: A virtual function that accepts an argument for the amount of the withdrawal. The function should subtract the argument from the balance. It should also increment the variable holding the number of withdrawals.

calcInt: A virtual function that updates the balance by calculating the monthly interest earned by the account, and adding this interest to the balance. This is performed by the following formulas:

$$\text{Monthly Interest Rate} = (\text{Annual Interest Rate} / 12)$$

$$\text{Monthly Interest} = \text{Balance} * \text{Monthly Interest Rate}$$

$$\text{Balance} = \text{Balance} + \text{Monthly Interest}$$

monthlyProc: A virtual function that subtracts the monthly service charges from the balance, calls the `calcInt` function, and then sets the variables that hold the number of withdrawals, number of deposits, and monthly service charges to zero.

Next, design a savings account class, derived from the generic account class. The savings account class should have the following additional member:

`status` (to represent an active or inactive account)

If the balance of a savings account falls below \$25, it becomes inactive. (The `status` member could be a flag variable.) No more withdrawals may be made until the balance is raised above \$25, at which time the account becomes active again. The savings account class should have the following member functions:

- withdraw:** A function that checks to see if the account is inactive before a withdrawal is made. (No withdrawal will be allowed if the account is not active.) A withdrawal is then made by calling the base class version of the function.
- deposit:** A function that checks to see if the account is inactive before a deposit is made. If the account is inactive and the deposit brings the balance above \$25, the account becomes active again. The deposit is then made by calling the base class version of the function.
- monthlyProc:** Before the base class function is called, this function checks the number of withdrawals. If the number of withdrawals for the month is more than 4, a service charge of \$1 for each withdrawal above 4 is added to the base class variable that holds the monthly service charges. (Don't forget to check the account balance after the service charge is taken. If the balance falls below \$25, the account becomes inactive.)

Next, design a checking account class, also derived from the generic account class. It should have the following member functions:

- withdraw:** Before the base class function is called, this function will determine if a withdrawal (a check written) will cause the balance to go below \$0. If the balance goes below \$0, a service charge of \$15 will be taken from the account. (The withdrawal will not be made.) If there isn't enough in the account to pay the service charge, the balance will become negative and the customer will owe the negative amount to the bank.
- monthlyProc:** Before the base class function is called, this function adds the monthly fee of \$5 plus \$0.10 per withdrawal (check written) to the base class variable that holds the monthly service charges.

Write a complete program that demonstrates these classes by asking the user to enter the amounts of deposits and withdrawals for a savings account and checking account. The program should display statistics for the month, including beginning balance, total amount of deposits, total amount of withdrawals, service charges, and ending balance.



NOTE: You may need to add more member variables and functions to the classes than those listed above.